

MODUL 10

PRAKTIKUM PEMROGRAMAN MULTIMEDIA & GAME

Procedural Maze Generation

Recursive Depth First Search Algoritm

Pada Modul ini, Mahasiswa akan mempelajari tentang Maze Generation Recursive Depth First Search Algoritm. Berikut adalah Langkah langkah implementasinya.

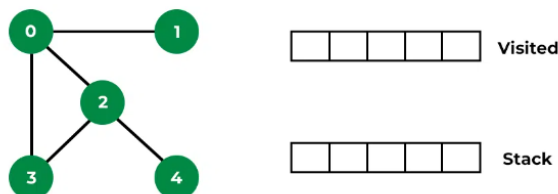
1. Logika Recursive Depth First Search

Depth First Search (DFS) atau Pencarian Pertama Kedalaman merupakan algoritma yang digunakan untuk menelusuri atau mencari struktur data pohon atau graf. Algoritma ini memulai penelusuran dari suatu simpul tertentu dan mengeksplorasi sejauh mungkin di setiap cabang sebelum melakukan backtracking.

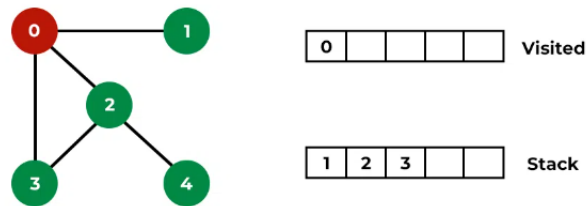
Konsep dasarnya adalah memulai dari suatu simpul tertentu, kemudian menjelajahi sejauh mungkin di setiap cabang sebelum melakukan backtracking. DFS bekerja dengan cara mempertahankan sebuah tumpukan (atau menggunakan rekursi) untuk melacak simpul-simpul yang akan dikunjungi.

Untuk lebih memahami DFS, perhatikan ilustrasi berikut.

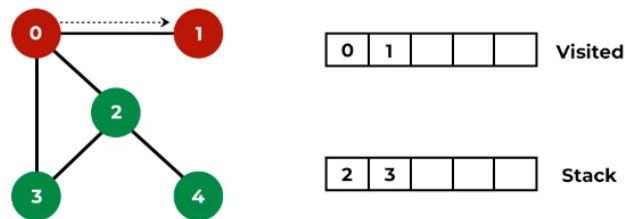
- Terdapat Graph dan table visited dan stack yang kosong.



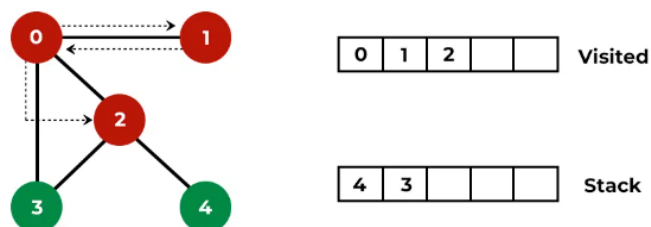
- Telusuri node 0 dan memasukan tetangga node 0 kedalam stack.



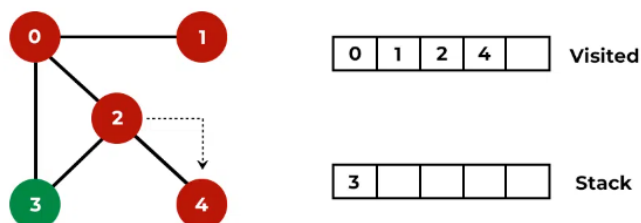
- Telusuri node 1.



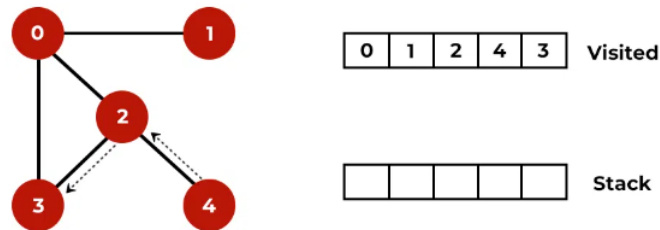
- Node 1 adalah node paling ujung dalam graph tersebut, oleh karena itu penelusuran akan Kembali ke node 0 dan dilanjutkan dengan penelusuran node 2. Node 4 ditambahkan pada stack karena merupakan tetangga node 2.



- Penelusuran dilanjutkan menuju node terdalam, yaitu node 4.



- Node 4 adalah node paling ujung, penelusuran Kembali ke node 2 dan dilanjutkan ke node 3. Dan penelusuran selesai karena seluruh node sudah ditelusuri.



2. Implementasi DFS pada Unity Engine

Berikut adalah Langkah Langkah implementasi Prims Algoritm pada unity.

- Buat Script baru Bernama **RecursiveDFS** didalam GameObject **MazeManager**.
- Pada Script **RecursiveDFS** akan mengimplementasi **Inheritance** dari script **MazeLogic**

```
Unity Script (1 asset reference) | 0 references
public class RecursiveDFS : MazeLogic
{
```

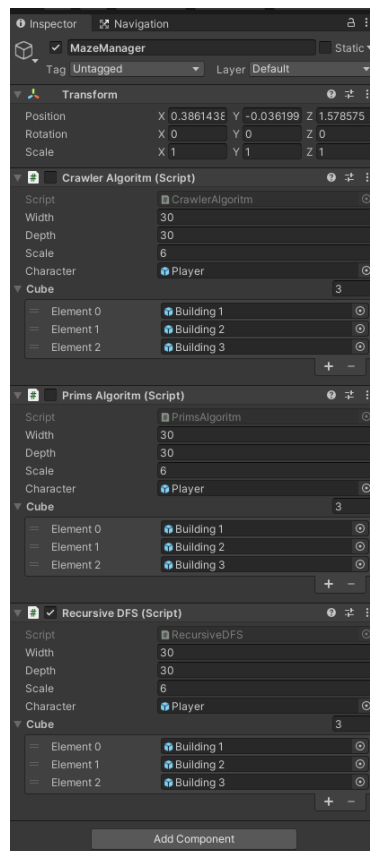
- Tulis Beberapa Baris Script dibawah, dan hapus Method **Start** dan **Update**.

```
2 references
public override void GenerateMaps()
{
    Generate(5, 5);
}

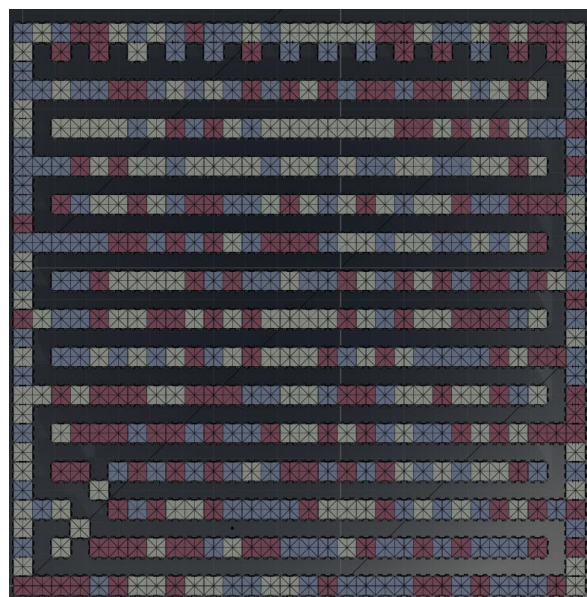
9 references
void Generate(int x, int z)
{
    if (CountSquareNeighbours(x, z) >= 2) return;
    map[x, z] = 0;

    Generate(x + 1, z);
    Generate(x - 1, z);
    Generate(x, z + 1);
    Generate(x, z - 1);
}
```

- Inisialisasi seperti gambar dibawah, aktifkan Recursive DFS lalu nonaktifkan Prims Algoritma dan CrawlerAlgoritma.



- Running Program, dan amati Maze yang tergenerate. Pola yang akan terbentuk kurang lebihnya seperti pada gambar dibawah, jika pola masih belum sama, lakukan pengecekan ulang!.



3. Implementasi Shuffle pada DFS.

Implementasi Shuffle pada DFS difungsikan untuk membentuk pola arah yang random, agar pola yang terbentuk tidak monoton seperti hasil bab 2. Shuffle ini akan diimplementasikan pada urutan index list **MapLocation**, seolah bermain kartu, dan setiap rekursinya akan selalu di acak, oleh karena itu index 0 tidak selalu menelusuri sisi kanan, dan berlaku pada index setelahnya. Berikut adalah implementasi kode Shuffle.

- Ketik penambahan index baru pada List **MapLocation**. 4 Value tersebut merupakan operasi arah penelusuran.

```
Unity Script (1 asset reference) | 0 references
public class RecursiveDFS : MazeLogic
{
    public List<MapLocation> directions = new List<MapLocation>() {
        new MapLocation(1,0),
        new MapLocation(0,1),
        new MapLocation(-1,0),
        new MapLocation(0,-1) };
}
```

- Buat Script baru Bernama **Extension**. Dan ketik beberapa baris kode berikut.

```
0 references
public static class Extensions
{
    private static System.Random rng = new System.Random();
    1 reference
    public static void Shuffle<T>(this IList<T> list)
    {
        int n = list.Count;
        while (n > 1)
        {
            n--;
            int k = rng.Next(n + 1);
            T value = list[k];
            list[k] = list[n];
            list[n] = value;
        }
    }
}
```

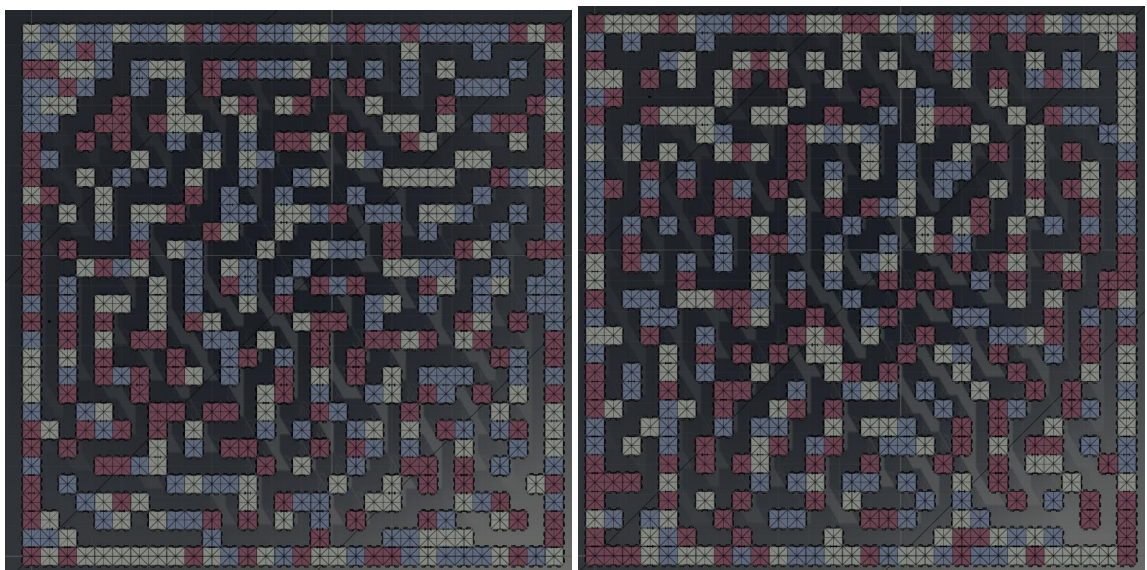
- Kembali ke Script **RecursiveDFS** dan sesuaikan method Generate.

```
9 references
void Generate(int x, int z)
{
    if (CountSquareNeighbours(x, z) >= 2) return;
    map[x, z] = 0;

    directions.Shuffle();

    Generate(x + directions[0].x, z + directions[0].z);
    Generate(x + directions[1].x, z + directions[1].z);
    Generate(x + directions[2].x, z + directions[2].z);
    Generate(x + directions[3].x, z + directions[3].z);
}
```

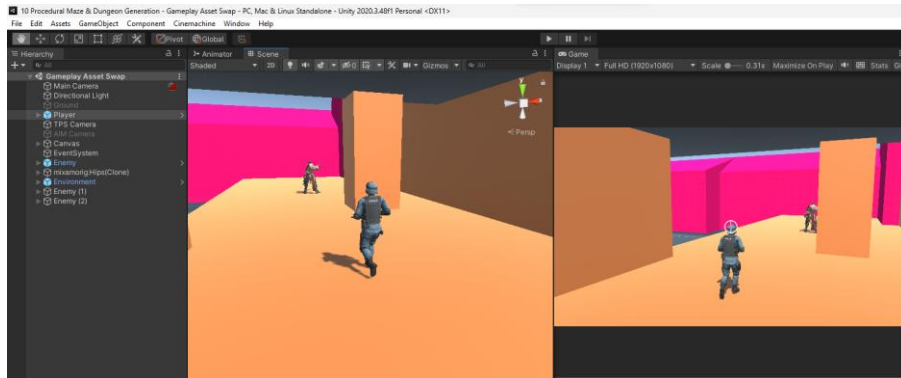
- Kembali ke Unity dan masuk kedalam Play Mode. Pola yang akan terbentuk kurang lebihnya seperti pada gambar dibawah, akan terlihat arah penelusuran yang lebih bervariasi. jika pola masih belum sama, lakukan pengecekan ulang!.



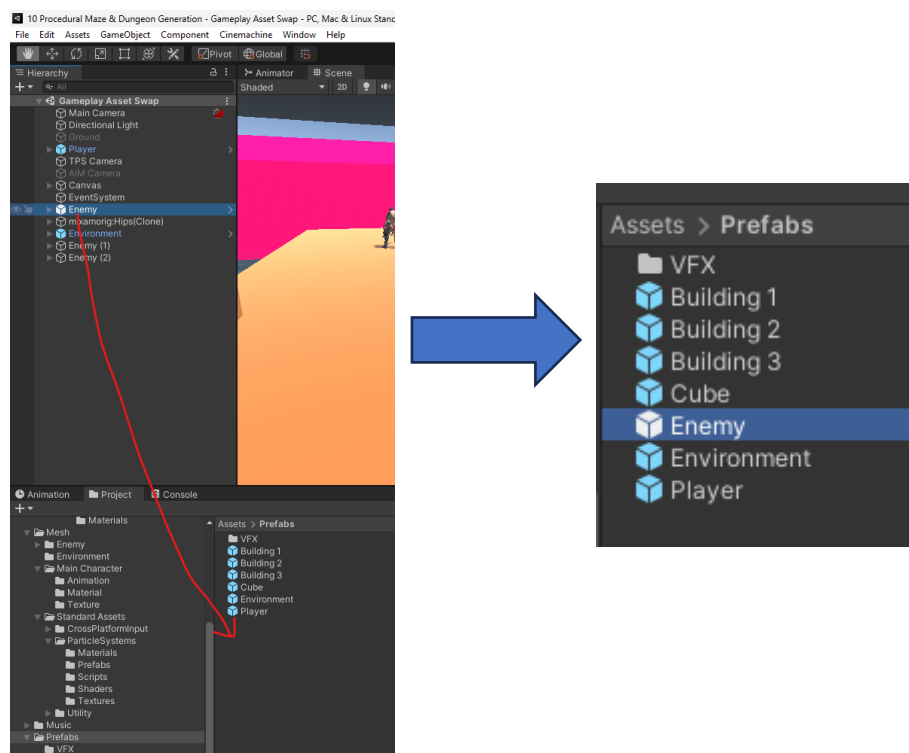
4. Place the Enemy Automatically With Script

Proses Selanjutnya adalah memasang enemy kedalam maze secara otomatis menggunakan script. Berikut adalah Langkah langkahnya.

- Buka Scene Gameplay (Scene yang terdapat Dummy Stage)



- Drag and Drop GameObject Enemy kedalam folder prefab (jika masih belum ada create Folder terlebih dahulu).



- Buka Script MazeLogic dan tuliskan 2 Variable Global baru dibawah ini.

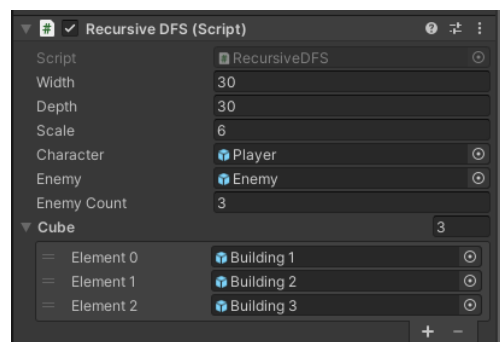
```
public int width = 30; //x length
public int depth = 30; //z length
public int scale = 6;
public GameObject character;
public GameObject Enemy;
public int EnemyCount = 3;
public List<GameObject> Cube; //Maze Wall
public byte[,] map;
GameObject[,] BuildingList;
```

- Tulis Method PlaceEnemy.

```
1 reference
public virtual void PlaceEnemy()
{
    int EnemySet = 0;
    for (int i = 0; i < depth; i++)
    {
        for (int j = 0; j < width; j++)
        {
            int x = Random.Range(0, width);
            int z = Random.Range(0, depth);
            if (map[x, z] == 0 && EnemySet != EnemyCount)
            {
                Debug.Log("placing Enemy");
                EnemySet++;
                Instantiate(Enemy, new Vector3(x * scale, 0, z * scale), Quaternion.identity);
            }
            else if (EnemySet == EnemyCount)
            {
                Debug.Log("already Placing All the Enemy");
                return;
            }
        }
    }
}
```

- Atur Method Start dan Inspector seperti dibawah ini.

```
Unity Message | 0 references
void Start()
{
    InitialiseMap();
    GenerateMaps();
    DrawMaps();
    PlaceCharacter();
    PlaceEnemy();
}
```



- Kembali ke Unity Scene Maze, dan masuk kedalam Play mode, jika pada maze sudah muncul enemy sejumlah 3, maka konfigurasi dan script sudah benar, jika belum lakukan pengecekan ulang!

5. Selesai Untuk Praktikum 10.

Praktikum 10 dikatakan selesai jika praktikan sudah menyelesaikan Maze Generation menggunakan algoritma Recursive Deep First Search, dan sudah memunculkan enemy didalam maze secara otomatis.

Tugas Modul 10

Tidak ada Tugas Untuk Modul Ke 10

Fokus kepada progressing ke 4 dan maju secara berkelompok ke dosen pengampu mata kuliah praktikum game.